Samsung Galaxy S10+ (Exynos) PMS Linux Server Guide

Complete Technical Manual: Bootloader Unlocking, Magisk Root, Chroot Linux, Stack Deployment & 24/7 Hardening

Document Overview & Hardware Specifications

Specification	Details for Samsung Galaxy S10+	Suitability for PMS
Model Variants	SM-G975F / SM-G975FD / SM-G975N (`beyond2lte`)	European / Global / Korean Exynos Models
Processor	Samsung Exynos 9820 (8nm LPP)	8 Cores (2x Mongoose M4 @ 2.73GHz + 2x Cortex-A75 @ 2.31GHz + 4x Cortex-A55 @ 1.95GHz)
Architecture	**ARM64** / **aarch64**	Fully supported by Node.js 20+, PostgreSQL, Redis, Nginx, Docker
Memory (RAM)	**8 GB** LPDDR4X (or 12 GB on Ceramic)	PMS Stack consumes ~750 MB total; **> 7 GB Headroom**
Storage	**128 GB** / **512 GB** / **1 TB** UFS 2.1	High-speed random I/O (faster than SATA SSDs or SD cards)
Power Backup	**4,100 mAh** Lithium-Ion Internal Battery	**Built-in Hardware UPS** (survives 6-10 hours of power outages)
Power Draw	**2.5W - 5.5W** (idle to load)	Negligible electricity cost (~$0.40/month)
Networking	Wi-Fi 6 (802.11ax) + USB 3.1 Type-C OTG	Supports Gigabit USB-C to Ethernet (RJ-45) adapters natively

1. Why the Rooting (Native Chroot) Method?

While non-rooted methods like Termux with PRoot exist, **the Rooting Method is the gold standard for turning an Android phone into a production server** for several crucial reasons:

1. **Native Linux System Calls (Zero Emulation Overhead):** PRoot intercepts and emulates system calls via `ptrace`, creating an I/O and CPU bottleneck. A rooted **chroot** runs directly on the native Linux kernel compiled into Android with 100% native execution speed.
2. **Privileged Network Ports:** Non-root environments cannot bind to ports below 1024 (e.g. port 80 for HTTP or port 443 for HTTPS). Root access allows direct binding to any port.
3. **Hardware Charge Limiting (Battery Bloat Prevention):** A phone connected to a charger 24/7 at 100% battery will develop pouch swelling (a serious fire hazard). Root access enables direct kernel-level charging control (stopping charge at 70% and resuming at 60%).
4. **True Unkillable Background Daemons:** Android OS aggressively kills background processes ("Phantom Process Killer"). Rooted background services initialized through Magisk init scripts cannot be terminated by Android's `ActivityManager`.
5. **Auto-Boot on Power Loss:** If the battery ever completely depletes and power returns, the phone can be configured to automatically boot and restart the entire PMS stack without pressing any buttons.

2. Phase 1: Bootloader Unlocking (Exynos S10+)

> **WARNING:** Unlocking the bootloader will permanently trip Samsung Knox (0x1), disable Samsung Pay/Pass/Secure Folder, and trigger a **full factory reset (all data will be wiped)**. Back up any personal files before proceeding.

Step 1.1: Enable Developer Options

1. On your S10+, go to **Settings > About phone > Software information**.
2. Tap **Build number** 7 times rapidly until you see *"Developer mode has been enabled"*.
3. Go back to **Settings > Developer options**.
4. Enable **USB debugging**.
5. Enable **OEM unlocking**.
6. *If "OEM unlocking" is missing:* Connect the phone to Wi-Fi, set the system date 10 days in the past, disable automatic updates, reboot, and check again. Samsung's "Vaultkeeper" requires internet connectivity to expose the toggle.

Step 1.2: Unlock the Bootloader via Download Mode

1. Power off the phone completely.
2. Press and hold **Volume Down + Bixby** simultaneously, and connect the phone to your PC via a USB cable.
3. The phone will boot into the blue **Warning** / **Download Mode** screen.
4. **Long press Volume Up** (Device Unlock Mode).
5. Press **Volume Up** once more to confirm bootloader unlock.
6. The phone will perform a factory data wipe and reboot.
7. Complete the initial setup wizard quickly (skip Google and Samsung account sign-ins). Connect to Wi-Fi.
8. Re-enable Developer Options and verify that **OEM unlocking** is greyed out and displays *"Bootloader already unlocked"*.

---

3. Phase 2: Rooting with Magisk (System-as-Root / SAR)

The Galaxy S10+ uses Android's System-as-Root partition layout. Magisk is installed by patching the stock firmware `AP` file.

Step 2.1: Download Stock Firmware

1. Check your exact model number in Settings (e.g., `SM-G975F`) and Region/CSC (e.g., `PAK`, `XSG`, `BTU`).
2. Download **Frija** or **Bifrost** on your PC to download the latest official One UI firmware for your CSC.
3. Extract the downloaded zip file. You will get 5 files:
4. `AP_G975F...tar.md5`
5. `BL_G975F...tar.md5`
6. `CP_G975F...tar.md5`
7. `CSC_OXM_G975F...tar.md5`
8. `HOME_CSC_OXM_G975F...tar.md5`

Step 2.2: Patch AP via Magisk App

1. Install the official **Magisk App** (v27.0 or newer) on your S10+ via APK.
2. Copy the stock `AP_G975F...tar.md5` file from your PC to the phone's Download folder.
3. Open Magisk, tap **Install** next to the Magisk card.
4. Choose **"Select and Patch a File"**, navigate to the AP file in Downloads, and tap it.
5. Magisk will unpack the boot/recovery images, patch the kernel, and generate a file named: `magisk_patched_[random_strings].tar` in your `Download/` folder.
6. Copy `magisk_patched_....tar` back to your PC.

Step 2.3: Flash Patched Firmware with Odin

1. Download **Odin v3.14.4** on your PC and install Samsung USB Drivers.
2. Put the S10+ into Download Mode (**Volume Down + Bixby + Plug in USB** -> Short press **Volume Up**).
3. Open Odin as Administrator. Verify the `ID:COM` box lights up blue/cyan.
4. Assign the slots:
5. **BL:** Select `BL_G975F...tar.md5`
6. **AP:** Select `magisk_patched_....tar`
7. **CP:** Select `CP_G975F...tar.md5`
8. **CSC:** Select `CSC_OXM_G975F...tar.md5`
9. In Odin's **Options** tab, ensure **Auto Reboot** and **F. Reset Time** are checked.
10. Click **Start**. Wait until Odin shows a green **PASS!** box (takes 3-5 minutes).
11. The phone will reboot. Open the Magisk app; it will prompt for *"Additional Setup"*. Tap **OK** and let the phone reboot one last time.
12. **Verification:** Open Termux or ADB shell, type `su`, and tap **Grant** on the Magisk Superuser prompt. Root access is now fully active!

---

4. Phase 3: 24/7 Hardware, Battery & Thermal Protection

Step 3.1: Install ACC (Advanced Charging Controller) to Prevent Battery Bloat

Keeping an S10+ plugged in 24/7 at 100% charge causes lithium pouch swelling and permanent destruction of the battery. We configure the kernel to cycle charge between 60% and 70%:

1. In the Magisk app, search and install the **BusyBox for Android NDK** module.
2. Install **VR-25's ACC (Advanced Charging Controller)** via Magisk or Termux:
   `bash su pkg install curl sh -c "$(curl -fsSL https://raw.githubusercontent.com/VR-25/acc/master/install.sh)"`
3. Configure ACC limits:
   `bash su -c acc 70 60`
4. *What this does:* Charging automatically cuts off at **70%** and resumes only when battery drops to **60%**. The battery never stays saturated at 4.35V, extending lifespan to 5+ years without swelling.

Step 3.2: Disable Android's Phantom Process Killer & Sleep

Android 12 kills background tasks that consume CPU when the screen is locked:
1. Connect via ADB from your PC (or run via `su` in Termux):
`bash su -c "/system/bin/device_config put activity_manager max_phantom_processes 2147483647" su -c "/system/bin/device_config set_sync_disabled_for_tests persistent"`
2. On the phone, go to **Settings > Apps > Termux / Linux Deploy**:
* Set **Battery** to **Unrestricted**.
3. Go to **Settings > Display**:
* Turn **Always On Display** completely **OFF**.
* Set Brightness to **Minimum**.

Step 3.3: Network & Physical Setup

- **Recommended Adapter:** Purchase a **USB-C Hub with Power Delivery (PD) + RJ-45 Ethernet** (brands like Ugreen, Anker, or Baseus).
- Connect the ethernet cable directly from your office router to the phone. Android will show an Ethernet icon (`<...>` or `<eth0>`) instantly without drivers.
- Reserve a **Static IP** on your router for the phone's MAC address (e.g. `192.168.1.150`).

---

5. Phase 4: Deploying Debian / Ubuntu via Rooted Chroot

We will deploy a full **Ubuntu 22.04 LTS (ARM64)** environment inside a native chroot container.

Step 4.1: Setup Linux Deploy

1. Install **Linux Deploy** (from GitHub releases or F-Droid).
2. Open Linux Deploy and tap the **Properties icon** (bottom right):
3. **Distribution:** `Ubuntu`
4. **Distribution suite:** `jammy` (Ubuntu 22.04 LTS)
5. **Architecture:** `arm64`
6. **Installation type:** `Directory`
7. **Installation path:** `/data/local/linux`
8. **User name:** `pms`
9. **User password:** `SetAStrongPassword123`
10. **Privileged users:** `root`
11. **DNS IP:** `1.1.1.1`
12. **Localization:** `en_US.UTF-8`
13. Check **Enable SSH** (SSH Port: `22`)
14. Tap the three dots menu at the top right and select **Install**.
15. Linux Deploy will partition the directory, download official Ubuntu ARM64 debs, build the root filesystem, and configure users (takes ~5 minutes).
16. Once it shows `<<< deploy`, tap **START** at the bottom.
17. Test SSH access from your PC:
    `bash ssh pms@192.168.1.150`

---

6. Phase 5: Installing the PMS Software Stack

Log in to your S10+ Ubuntu environment via SSH:

```
ssh pms@192.168.1.150
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl wget git build-essential ufw htop rsync
```

Step 6.1: Install Node.js 20 LTS (ARM64)

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
sudo npm install -g pnpm pm2
node -v # Returns v20.x.x (aarch64)
```

Step 6.2: Install & Configure PostgreSQL 16

```
sudo apt install -y postgresql postgresql-contrib
sudo systemctl enable postgresql
sudo systemctl start postgresql

# Create PMS database and user
sudo -u postgres psql << EOF
CREATE USER pms WITH PASSWORD 'YourSecurePmsDbPassword';
CREATE DATABASE pms OWNER pms;
GRANT ALL PRIVILEGES ON DATABASE pms TO pms;
\q
EOF
```

Performance Tuning for Exynos 9820 & 8GB RAM

Edit `/etc/postgresql/16/main/postgresql.conf`:

```
shared_buffers = 1GB
effective_cache_size = 3GB
maintenance_work_mem = 256MB
checkpoint_completion_target = 0.9
wal_buffers = 16MB
default_statistics_target = 100
random_page_cost = 1.1
effective_io_concurrency = 200
work_mem = 16MB
min_wal_size = 1GB
max_wal_size = 4GB
max_connections = 100
```

Restart PostgreSQL:

```
sudo systemctl restart postgresql
```

Step 6.3: Install & Configure Redis

```
sudo apt install -y redis-server
sudo systemctl enable redis-server
sudo systemctl start redis-server
```

---

7. Phase 6: Deploying the Property Management System Code

Step 7.1: Clone and Build

```
sudo mkdir -p /var/www/pms
sudo chown -R pms:pms /var/www/pms
cd /var/www/pms

# Transfer project files or clone repository
git clone <YOUR_GIT_REPO_URL> .
pnpm install

# Run database migration & prisma generator
pnpm --filter api prisma:generate
pnpm --filter api prisma:migrate

# Build production bundles
pnpm run build
```

Step 7.2: Production Environment Configuration (`.env`)

Create `/var/www/pms/.env`:

```
NODE_ENV=production
DATABASE_URL="postgresql://pms:YourSecurePmsDbPassword@127.0.0.1:5432/pms?schema=public"
REDIS_URL="redis://127.0.0.1:6379"

# API Port & Binding
API_PORT=3001
API_HOST=0.0.0.0

# Security
JWT_SECRET=super_long_production_jwt_secret_key_s10_server
DEFAULT_TENANT_ID=t_local

# Storage (Local file storage on fast UFS flash)
STORAGE_MODE=local
LOCAL_STORAGE_DIR=/var/www/pms/storage
```

Step 7.3: PM2 Process Management

Create `/var/www/pms/ecosystem.config.js`:

```
module.exports = {
  apps: [
    {
      name: 'pms-api',
      cwd: '/var/www/pms/apps/api',
      script: 'dist/main.js',
      instances: 1,
      exec_mode: 'fork',
      max_memory_restart: '500M',
      env: {
        NODE_ENV: 'production',
        PORT: 3001
      }
    },
    {
      name: 'pms-worker',
      cwd: '/var/www/pms/apps/worker',
      script: 'dist/main.js',
      instances: 1,
      exec_mode: 'fork',
      max_memory_restart: '300M',
      env: {
        NODE_ENV: 'production'
      }
    },
    {
      name: 'pms-web',
      cwd: '/var/www/pms/apps/web',
      script: 'node_modules/next/dist/bin/next',
      args: 'start -p 3000 -H 0.0.0.0',
      instances: 1,
      exec_mode: 'fork',
      max_memory_restart: '400M',
      env: {
        NODE_ENV: 'production',
        PORT: 3000
      }
    }
  ]
};
```

Start the services:

```
pm2 start ecosystem.config.js
pm2 save
```

---

8. Phase 7: Automated Boot & Recovery (Surviving Restarts)

If the phone ever reboots, we want the Linux chroot, PostgreSQL, Redis, and PMS to boot automatically without any manual intervention.

Step 8.1: Magisk Boot Script

On Android, Magisk executes scripts placed in `/data/adb/service.d/` on boot:
Create `/data/adb/service.d/01_pms_autostart.sh` via root shell:

```
su
cat << 'EOF' > /data/adb/service.d/01_pms_autostart.sh
#!/system/bin/sh
# Wait until system has booted
while [ "$(getprop sys.boot_completed)" != "1" ]; do
  sleep 3
done

# Keep CPU online and disable aggressive power management
echo "performance" > /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor 2>/dev/null
echo 2147483647 > /proc/sys/fs/file-max

# Start Linux Deploy Container
/data/data/ru.meefik.linuxdeploy/files/bin/linuxdeploy start -m

# Wait for chroot mount
sleep 5

# Execute chroot startup services
chroot /data/local/linux /bin/bash -c "
  service postgresql start
  service redis-server start
  su - pms -c 'pm2 resurrect'
"
EOF

chmod +x /data/adb/service.d/01_pms_autostart.sh
```

Step 8.2: Auto Power-On when Plugged into Charger

If the phone battery ever drains completely to 0% and shuts off, Samsung phones normally enter offline battery charging mode. We can force the phone to power on automatically when power is restored:

```
su
# On Samsung Exynos, modify the lpm (Low Power Mode) binary
mount -o remount,rw /system
if [ -f /system/bin/lpm ]; then
  mv /system/bin/lpm /system/bin/lpm.bak
  cat << 'EOF' > /system/bin/lpm
#!/system/bin/sh
/system/bin/reboot
EOF
  chmod 755 /system/bin/lpm
fi
```

Result: As soon as electrical power returns to the charger, the phone boots up immediately!

---

9. Phase 8: Remote Access (Cloudflare Tunnels) & Google Drive Backups

Step 9.1: Remote Access Without Port Forwarding (Cloudflare Tunnel)

You do not need a static public IP or open router ports. Use Cloudflare Tunnel on ARM64:

```
# Inside the S10+ Ubuntu environment:
curl -L --output cloudflared.deb https://github.com/cloudflare/cloudflared/releases/latest/download/
cloudflared-linux-arm64.deb
sudo dpkg -i cloudflared.deb

# Authenticate with Cloudflare
cloudflared tunnel login
cloudflared tunnel create pms-server

# Route traffic to your domain (e.g. pms.yourdomain.com -> 127.0.0.1:3000)
cloudflared tunnel route dns pms-server pms.yourdomain.com
sudo cloudflared service install <YOUR_TUNNEL_TOKEN>
```

Your PMS is now securely accessible worldwide over HTTPS with DDoS protection!

Step 9.2: Automated Nightly Backup to Google Drive

Using the existing repository backup integration:
1. Install `rclone`:
`bash sudo apt install -y rclone rclone config # Link your Google Drive account as 'gdrive'`
2. Setup a nightly cron job:
`bash crontab -e # Add nightly backup at 2:00 AM: 0 2 * * * /var/www/pms/backup-to-gdrive.sh >> /var/log/pms-backup.log 2>&1`

---

10. Operational Summary & Maintenance Checklist

Task	Frequency	Command / Method
Check PMS Status	Weekly	`pm2 status`
View Live Logs	As needed	`pm2 logs pms-api` or `pm2 logs pms-web`
Check Battery Health	Monthly	`su -c acc -i` (Verify temp < 38°C and capacity % is stable)
Verify Backups	Weekly	Check `gdrive:PMS_Backups` on Google Drive
PostgreSQL Vacuum	Monthly	`sudo -u postgres vacuumdb -z pms`

Document prepared for Property Management Suite (PMS) infrastructure.